

Organização e Arquitetura de Computadores

Circuitos Aritméticos

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

Nesta aula...

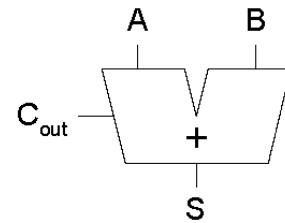
Application Software	>" he wor
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Introdução

- **Blocos digitais**
 - Portas lógicas, multiplexadores, decodificadores, registradores, circuitos aritméticos, ...
- **Utilizamos de maneira hierárquica para construir as estruturas lógicas da microarquitetura**
- **Na aula de hoje, as bases da ALU!**

Somador de 1 bit

Half Adder

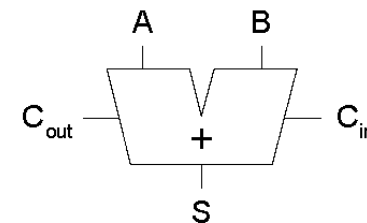


A	B	C _{out}	S
0	0		
0	1		
1	0		
1	1		

S =

C_{out} =

Full Adder



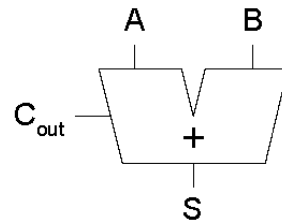
C _{in}	A	B	C _{out}	S
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

S =

C_{out} =

Somador de 1 bit

Half Adder

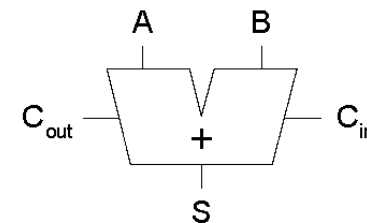


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S =$

$C_{out} =$

Full Adder



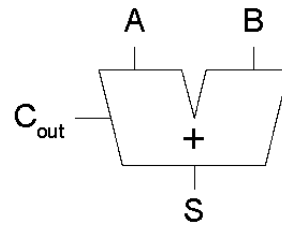
C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S =$

$C_{out} =$

Somador de 1 bit

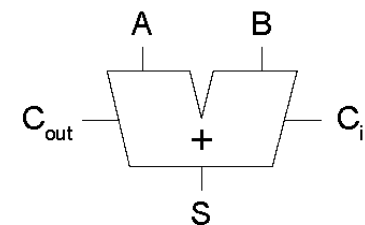
Half Adder



A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$
$$C_{out} = AB$$

Full Adder



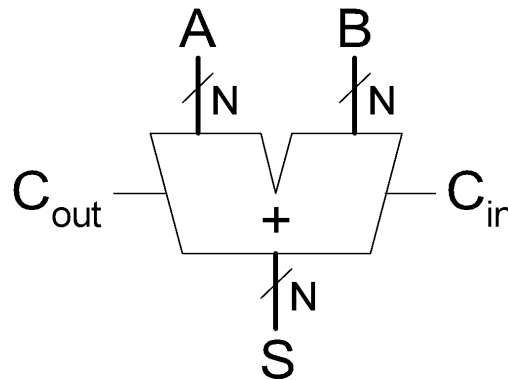
C _{in}	A	B	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

Somadores Multibit

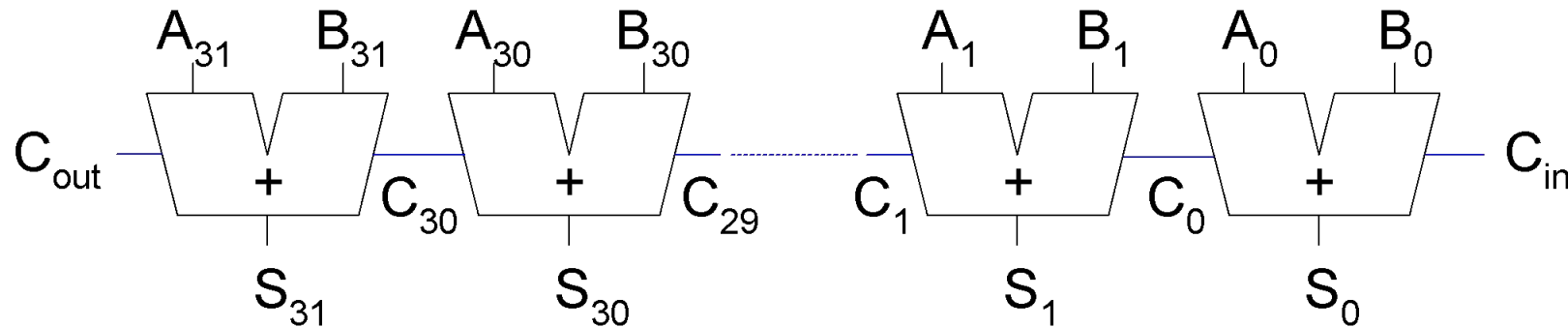
- Types of carry propagate adders (CPAs):
 - Ripple-carry (lento)
 - Carry-lookahead (rápido)
 - Prefixo (mais rápido)
- Carry-lookahead e somador de prefixo mais rápidos para maiores tamanhos de palavra, mas precisam de mais hardware

Símbolo



Somador Ripple-Carry

- Conecta somadores de 1 bit em sequência
- Carry passa pela sequência inteira
- Desvantagem: **lento**
 - Por que?



Somador Carry-Lookahead

- Computa carry out (C_{out}) para blocos de k bits usando sinais de *geração* e *propagação*
- **Definições:**
 - Coluna i produz carry out *gerando* um carry out ou *propagando* o carry in para o carry out
 - Sinais de Geração (G_i) e propagação (P_i) para cada coluna:
 - Coluna i vai gerar carry out se A_i AND B_i são ambos 1.

$$G_i = A_i B_i$$

- Coluna i vai propagar o carry in para o carry out se A_i OR B_i é 1.

$$P_i = A_i + B_i$$

- O carry out da coluna i (C_i) é:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1}$$

Somador Carry-Lookahead

- **Exemplo:** blocos de 4 bits ($G_{3:0}$ and $P_{3:0}$) :

$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

$$P_{3:0} = P_3 P_2 P_1 P_0$$

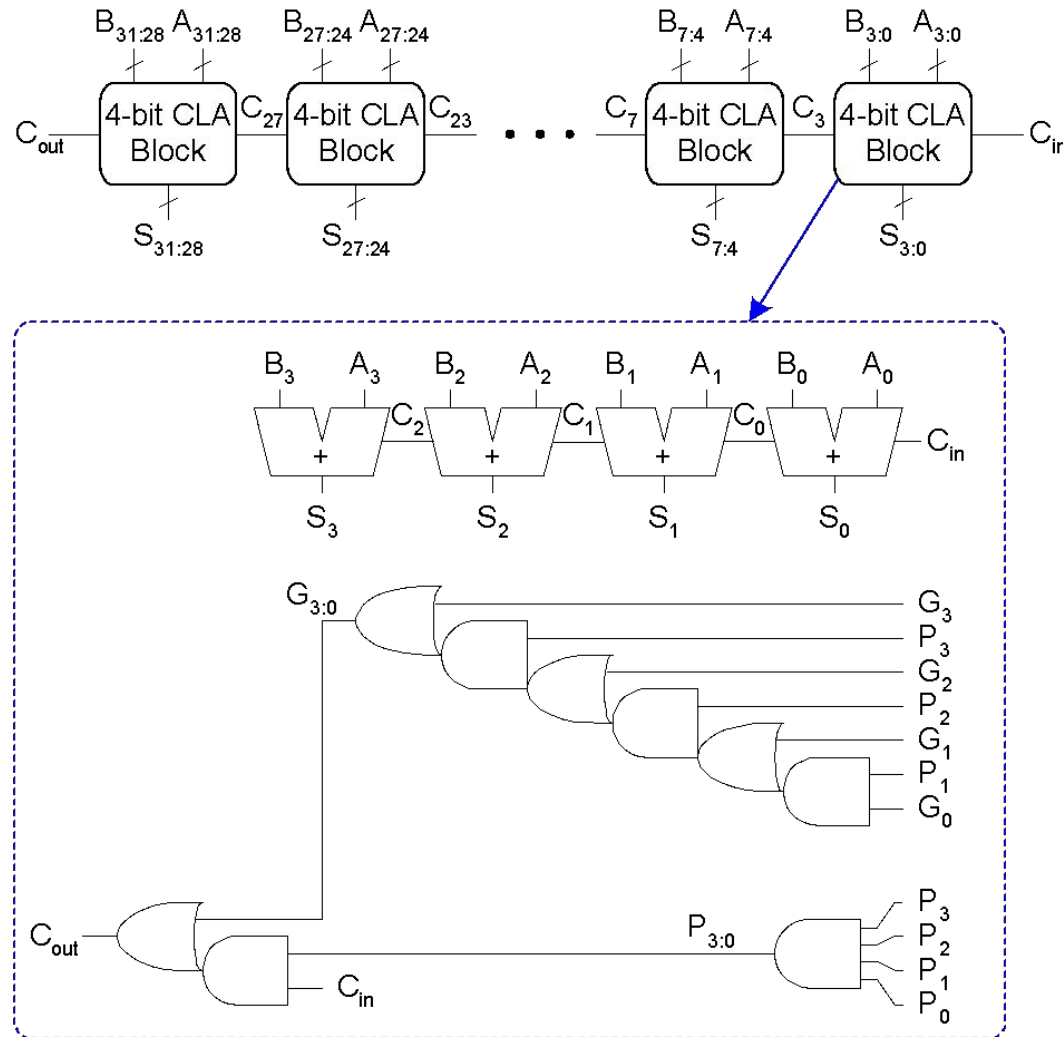
- **Generalizando,**

$$G_{i:j} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} G_j))$$

$$P_{i:j} = P_i P_{i-1} P_{i-2} P_j$$

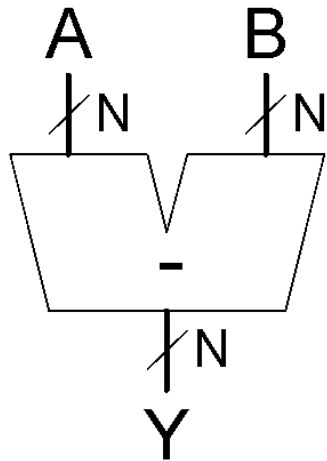
$$C_i = G_{i:j} + P_{i:j} C_{j-1}$$

CLA de 32 bits com blocos de 4 bits

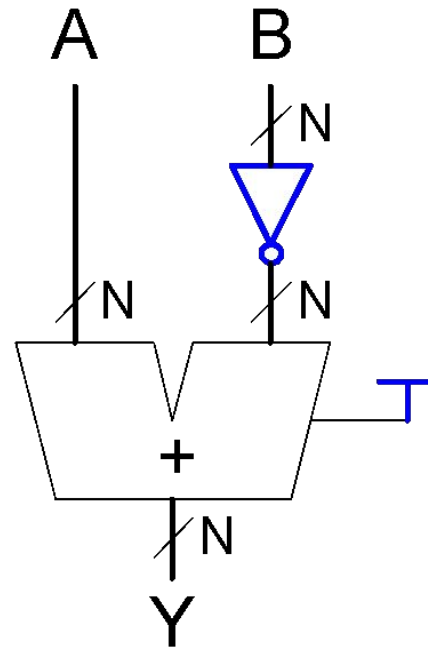


Subtractor

Symbol

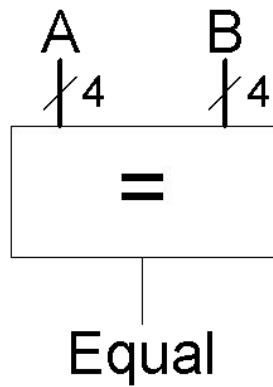


Implementation

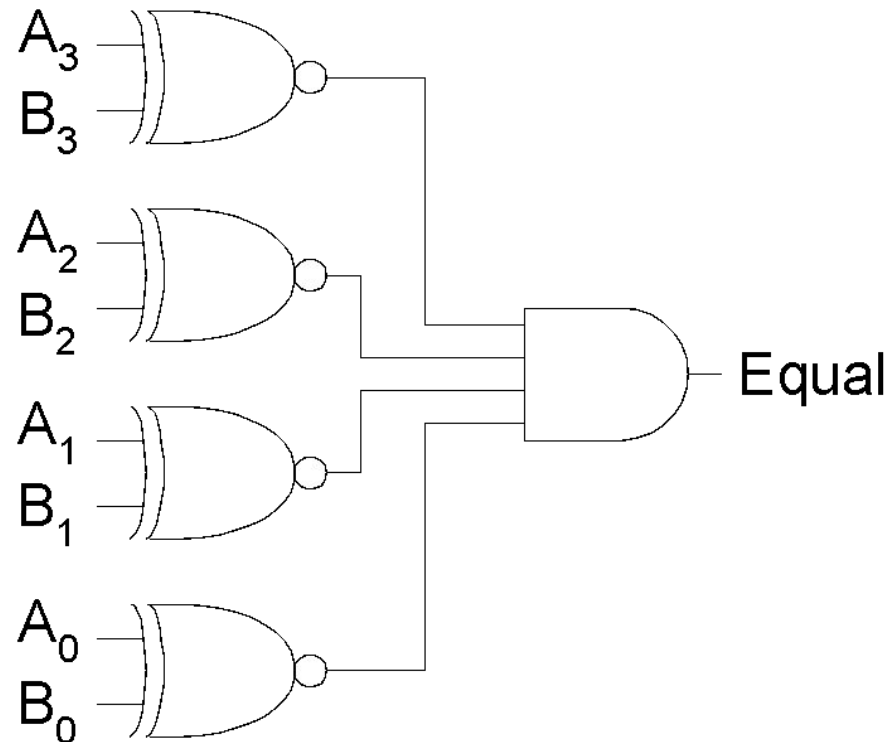


Comparator: Igualdade

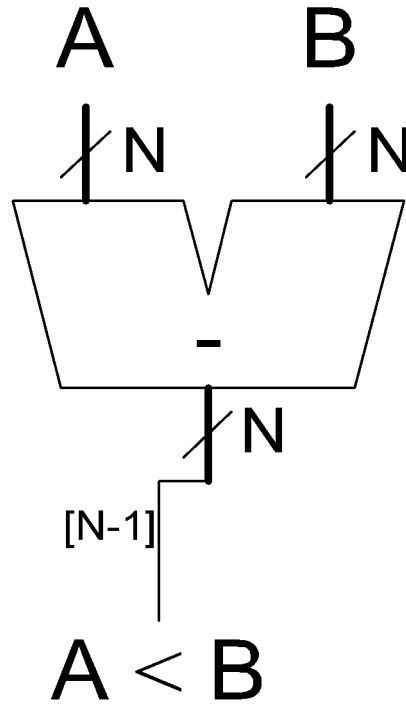
Symbol



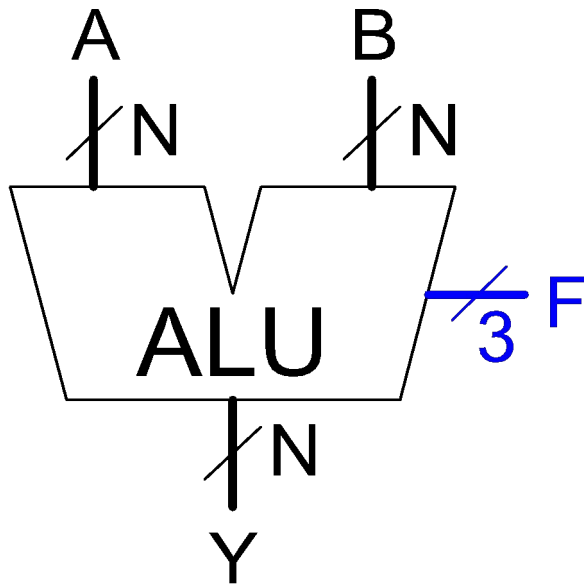
Implementation



Comparador: menor que

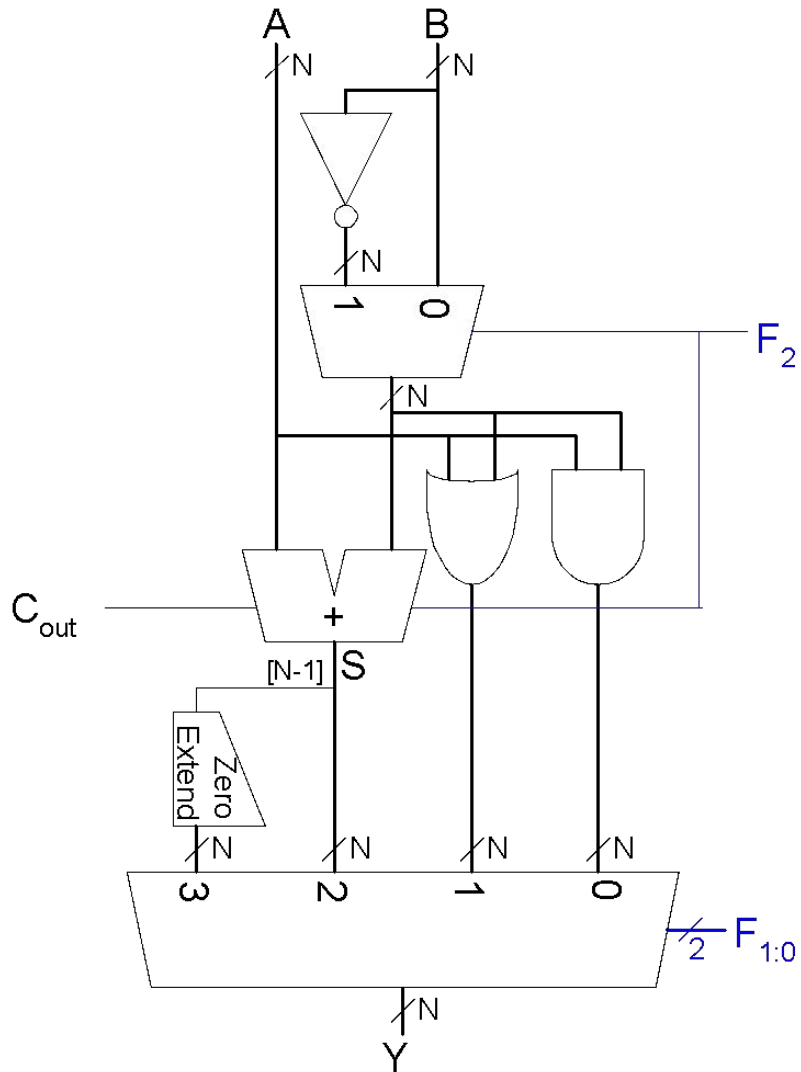


Unidade Lógica Aritmética (ALU) simples



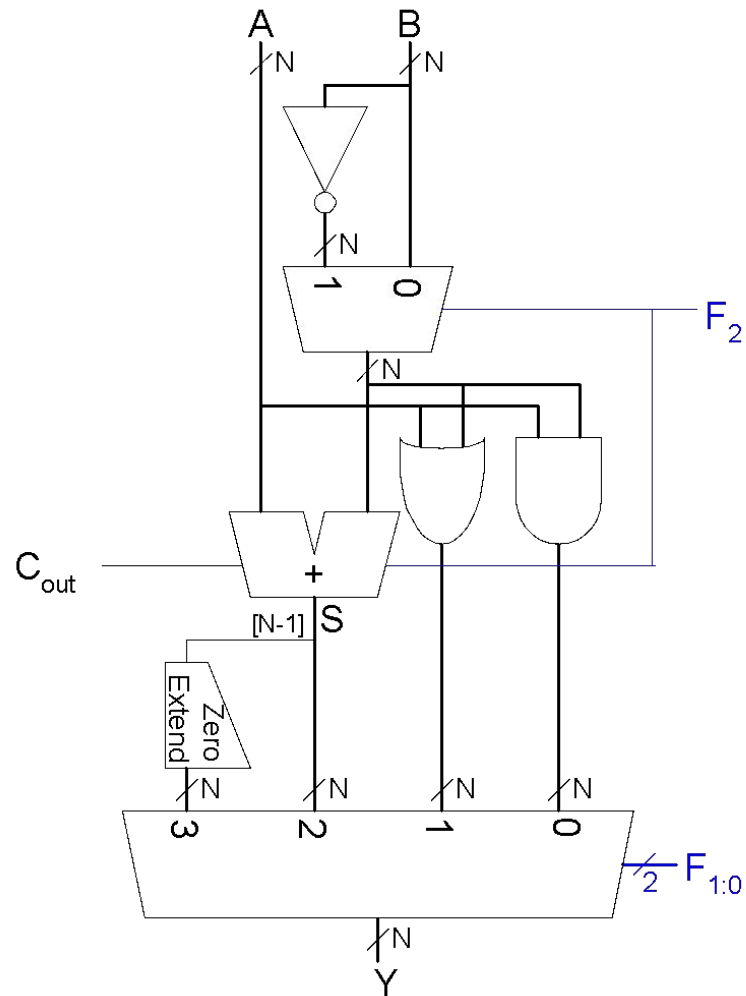
$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A - B
111	SLT

Design da ALU simples



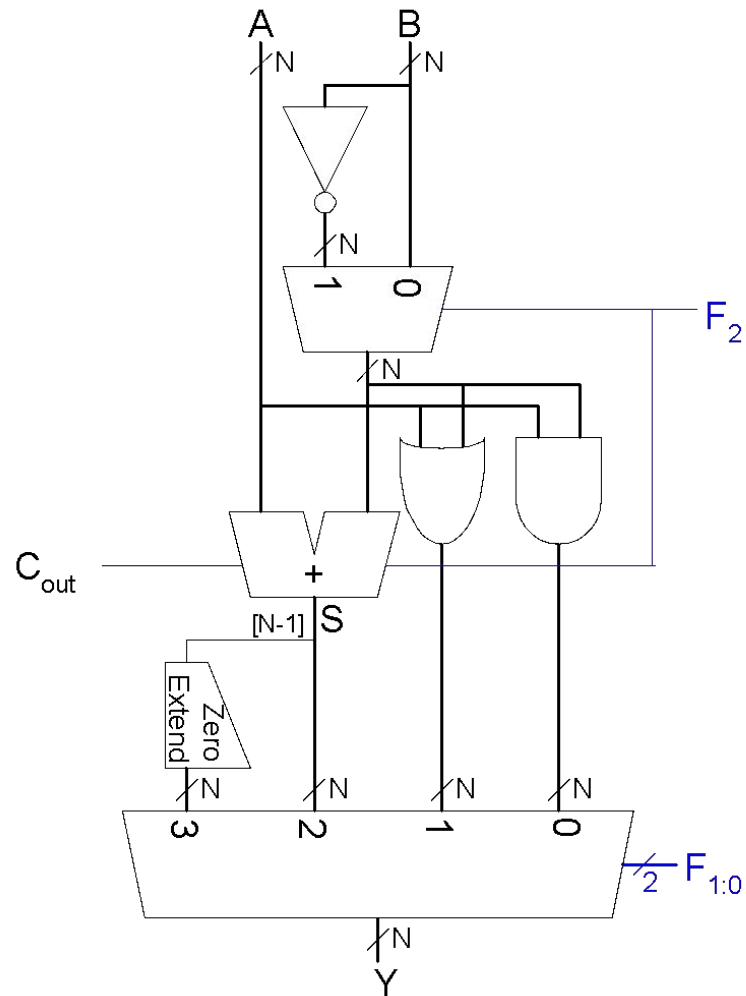
$F_{2:0}$	Function
000	$A \text{ AND } B$
001	$A \text{ OR } B$
010	$A + B$
011	not used
100	$A \text{ AND } \overline{B}$
101	$A \text{ OR } \overline{B}$
110	$A - B$
111	SLT

Exemplo Set Less Than (SLT)



- Configura ALU de 32 bits para SLT:
 - $A = 25$ e $B = 32$

Exemplo Set Less Than (SLT)



- Configura ALU de 32 bits para SLT:
 - $A = 25$ e $B = 32$
 - $A < B$, então Y deve ser representação de 32 bits do número 1 (0x00000001)
 - $F_{2:0} = 111$
 - $F_2 = 1$ (somador funciona como subtrator), então $25 - 32 = -7$
 - -7 tem 1 no seu bit menos significativo ($S_{31} = 1$)
 - $F_{1:0} = 11$ multiplexador escolhe $Y = S_{31}$ (estendendo zeros) = 0x00000001.